Construction d'un arbre hiérarchique hybride

État d'avancement méthodologique — résultats en cours de validation

Sommaire

Point de départ

Question 1 — Quelle famille de méthode ?

Question 2 — Dans quel ordre ?

Question 3 — Que contient un nœud ?

Question 4 — Sommes-nous vraiment nouveaux ?

Question 5 — Quel terrain et quel outil ?

État d'avancement — à l'essai

Point de départ

Le pipeline général

Maquette → Détection → **Arbre** → Code

La qualité du code final dépend presque entièrement de la qualité de l'arbre intermédiaire — c'est l'étape la plus fragile, celle que nous attaquons.

Chaque décision qui suit répond à une question précédente — suivons le fil.

Question 1 — Quelle famille de méthode ?

Le paysage des méthodes UI-to-code

- Systèmes à base de règles (REMAUI)
- Deep learning bout-en-bout (Pix2Code, Sketch2Code)
- Apprentissage par renforcement (Soselia et al.)
- Prompting direct de gros VLM (Design2Code)
- Hybride vision + deep learning (Bo et al.)

Pourquoi un hybride vision par ordinateur classique + MLLM

- **vs règles** : nous partons d'une image brute, sans métadonnée
- **vs deep learning bout-en-bout** : notre arbre reste explicite et inspectable à chaque étape
- **vs prompting direct de VLM** : notre géométrie est garantie par calcul, jamais par prédiction
- **vs hybrides existants** (DCGen, ScreenCoder, VSA) : chez eux, Le transformeur/MLLM de VSA, le MLLM de ScreenCoder intervient encore dans la structuration elle-même

→ Reste à trancher : dans quel ordre combiner calcul et MLLM ?

Question 2 — Dans quel ordre ?

Deux ordres possibles, deux exemples vérifiés

DesignCoder (arXiv :2506.13663) — le MLLM groupe en premier :

> *« Group the UI elements [...] into UI components. Output a complete hierarchical tree. »*

ScreenCoder (arXiv :2507.22827) — le MLLM positionne directement :

> *« queried with prompts such as 'Where is the sidebar ?' [...] returns [...] bounding boxes »*

Notre choix : l'inverse, assumé

Étape 1
via la vision par ordinateur classique
construit l'arbre complet

Étape 2
Le MLLM étiquette
les nœuds déjà construits

Le MLLM ne modifie jamais une position — il attache seulement un nom.

L'utilisation des outils de vision par ordinateur classique garantit une structure fiable et déterministe ; les erreurs de raisonnement du MLLM ne peuvent plus contaminer la géométrie.

→ Reste à définir : que doit contenir un nœud de cet arbre ?

Image d'une maquette avant la detection via la vision par ordinateur classique

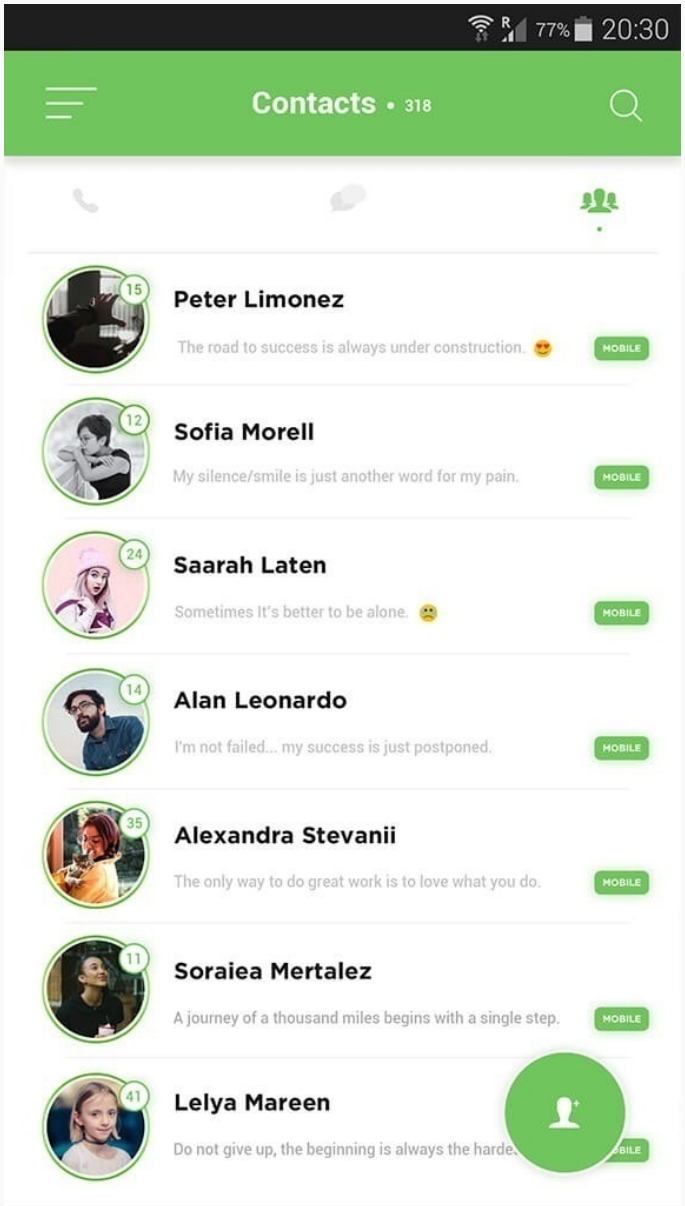
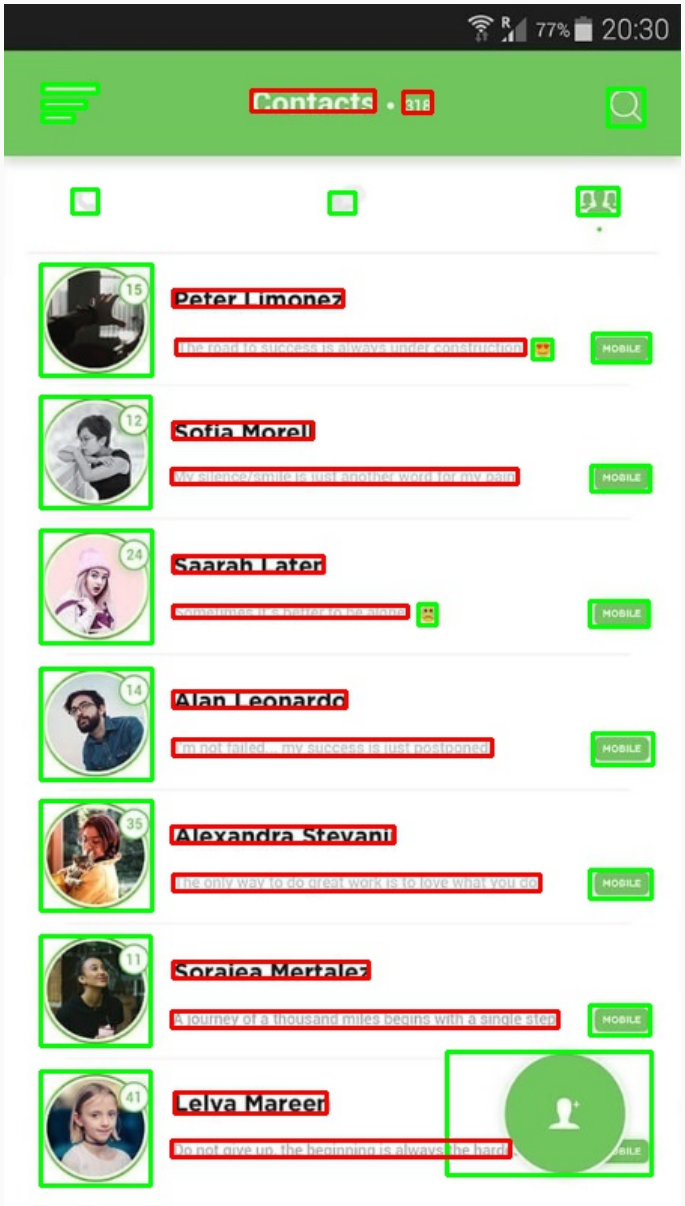


Image d'une maquette apres la detection via la vision par ordinateur classique



Question 3 — Que contient un nœud ?

Huit champs, quatre origines

Champ	Origine	Source qui justifie
id	Neutre	Zhang-Shasha (1989) — Tree Edit Distance
bbox	Détecteur	Universel (UIED, Screen Parsing)
type grossier	Détecteur	Consommé par Gestalt (clustering)
parent	Géométrie	Remontée et fusion
children (ordonné)	Géométrie	Zhang-Shasha — ordre significatif
type géométrique	Géométrie	Prototype2Code, ReverseORC
label sémantique	MLLM	Contribution centrale
hash	Géométrie	VSA (2025) — motif discovery

Question 4 — Sommes-nous vraiment nouveaux ?

Un vide de recherche à nuancer honnêtement

VSA (arXiv :2512.20034, déc. 2025) sépare déjà : structure → motifs → code.

Mais leur étape de structure utilise « *a spatial-aware transformer* » — un réseau de neurones.

→ Sur quel terrain et avec quel outil concret allons-nous appliquer tout cela ?

Question 5 — Quel terrain et quel outil ?

Le choix du détecteur

Screen Parsing (Wu et al., UIST 2021) — écarté : *« trained [...] on two mobile UI datasets : AMP [...] and RICO »*

ImageCLEF DrawnUI 2021 — écarté : entraîné sur du web, mais aucun poids ni code public trouvé.

UIED (Xie et al., ESEC/FSE 2020) — retenu : détection purement algorithmique, position + type grossier, sans réseau de neurones pour le sens.

Pourquoi Gestalt seul ne suffit pas sur le web

Gestalt (Xie et al., 2022, arXiv :2206.10352) regroupe les éléments en blocs naturels, sans besoin d'espace vide.

Évalué sur 1091 captures Android (Rico) + 20 prototypes Figma — et **limité à 2-3 niveaux de profondeur**.

Suffisant pour du mobile, généralement plus plat. Mais nos maquettes web présentent une profondeur DOM **moyenne de 17 niveaux** (min 2, max 40 — DCGen, Table 1, 348 pages).

Nous complétons donc Gestalt avec XY-Cut, appliqué récursivement à l'intérieur de chaque groupe, pour atteindre la vraie profondeur.

La chaîne complète

Détecteur (UIED)

↓ *position + type grossier*

Gestalt

↓ *regroupe en 2-3 niveaux (parent/children partiels)*

XY-Cut (récursif)

↓ *creuse jusqu'à la vraie profondeur*

MLLM

↓ *remplit label sémantique*

Arbre complet, annoté

État d'avancement — à l'essai

Important : nous testons encore, aucun résultat validé

- Installation testée : Google Colab, puis local (WSL) suite à un blocage Windows sur les bibliothèques compilées
- Deux incompatibilités de code corrigées (bibliothèques ayant évolué depuis 2022)
- Détection brute validée sur une maquette mobile de test
- Biais identifié sur une capture web : navigateur et barre des tâches inclus par erreur — à corriger

XY-Cut reste à écrire ; aucune mesure quantitative (Tree Edit Distance, TreeBLEU) n'a encore été réalisée.

Prochaines étapes

1. Refaire les tests sur de vraies captures de pages web complètes
2. Confirmer l'effet de `recognize_layout()` sur la fragmentation du texte
3. Écrire l'algorithme XY-Cut nous-mêmes
4. Mesurer quantitativement une fois le pipeline complet fonctionnel

Merci — questions et retours bienvenus